LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 1

ANDROID BASIC WITH COMPOSE

Oleh:

Noor Khalisa NIM. 2410817220012

PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 1

Laporan Praktikum Pemrograman Mobile Modul 1: Android Basic with Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Noor Khalisa
NIM : 2410817220012

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
B. Source Code 9
C. Output Program 14
D. Pembahasan 19
E. Essay 24
F. Daftar Pustaka 28
TAUTAN GIT 29

DAFTAR GAMBAR

Gambar 1. Tampilan Awal Aplikasi 6
Gambar 2. Tampilan Dadu Setelah Di-Roll 7
Gambar 3. Tampilan Roll Dadu Double 8
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose 14
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose 15
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose 16
Gambar 7. Screenshot Hasil Jawaban Soal 1 XML 17
Gambar 8. Screenshot Hasil Jawaban Soal 1 XML 18
Gambar 9. Screenshot Hasil Jawaban Soal 1 XML 19

DAFTAR TABEL

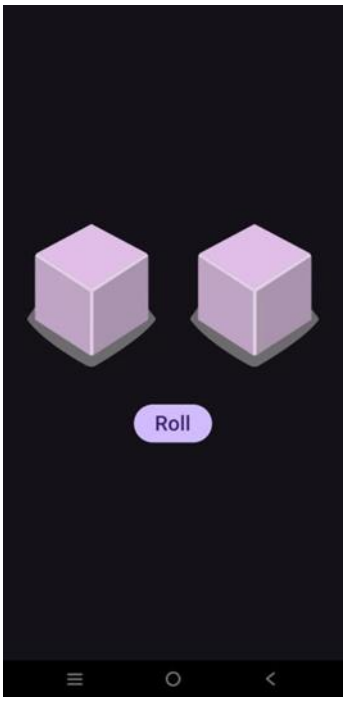
Tabel 1. Source Code MainActivity.kt (Compose) 9
Tabel 2. Source Code MainActivity.kt (XML) 11
Tabel 3. Source Code activity_main.xml (XML) 12

SOAL 1

Soal Praktikum:

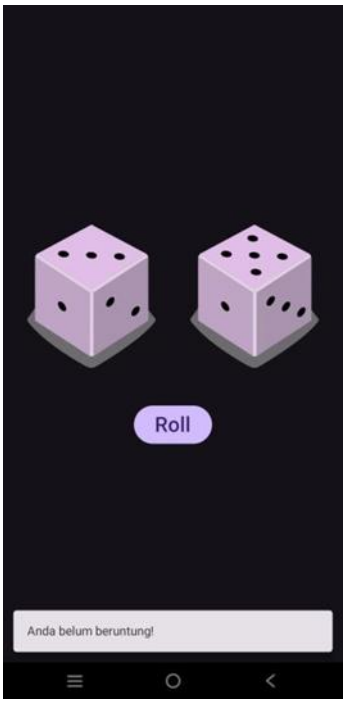
Buatlah sebuah aplikasi yang dapat menampilkan 2 buah dadu yang dapat berubah-ubah tampilannya pada saat user menekan tombol "Roll". Aturan aplikasi yang akan dibangun adalah sebagaimana berikut:

1. Tampilan awal aplikasi setelah dijalankan akan menampilkan 2 buah dadu kosong seperti dapat dilihat pada Gambar 1.



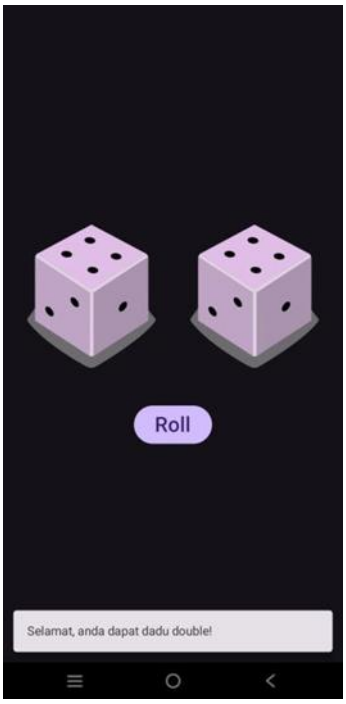
Gambar 1. Tampilan Awal Aplikasi

2. Setelah user menekan tombol "Roll" maka masing-masing dadu akan memperlihatkan sisi dadunya dengan angka antara 1 s/d 6. Apabila user mendapatkan nilai dadu yang berbeda antara Dadu 1 dengan Dadu 2, maka aplikasi akan menampilkan pesan "Anda belum beruntung!" seperti yang dapat dilihat pada Gambar 2.



Gambar 2. Tampilan Dadu Setelah Di-Roll

3. Apabila user mendapatkan nilai dadu yang sama antara Dadu 1 dan Dadu 2 atau nilai double, maka aplikasi akan menampilkan pesan “Selamat, anda dapat dadu double!” seperti yang dapat dilihat pada Gambar 3.



Gambar 3. Tampilan Roll Dadu Double

4. Buatlah aplikasi tersebut menggunakan XML dan Jetpack Compose.
5. Upload aplikasi yang telah anda buat ke dalam repository GitHub ke dalam **folder Modul 1 dalam bentuk Project**. Jangan lupa untuk melakukan **Clean Project** sebelum mengupload pekerjaan anda pada repository.
6. Untuk gambar dadu dapat didownload pada link berikut: https://drive.google.com/file/d/14V3qXGdFnuoYN4AGd_9SgFh8kw8X9ySm/view?usp= sharing.

Additional:

ESSAY

1. Riset dan *cite* artikel/jurnal/buku mengenai Imperative Programming dan Declarative Programming beserta penerapannya di pembuatan UI. Buatkan opini paradigma apa yang kalian prefer berdasarkan riset yang sudah dilakukan.
2. Apa yang membedakan OOP pada kotlin dengan OOP pada java?
3. Sebutkan, dan jelaskan mengenai SOLID Principles
4. Apa perbedaan antara Class dan Data Class pada Kotlin?

B. Source Code

Tabel 1. Source Code MainActivity.kt (Compose)

```
package com.example.modul1compose

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.unit.dp
import com.example.modul1compose.ui.theme.Modul1ComposeTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            Modul1ComposeTheme{
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    DiceRollerApp()
                }
            }
        }
    }
}

@Composable
fun DiceRollerApp() {
    var dice1 by remember { mutableIntStateOf(0) }
    var dice2 by remember { mutableIntStateOf(0) }
    var resultMessage by remember { mutableStateOf("") }

    fun rollDice() {
        dice1 = (1..6).random()
        dice2 = (1..6).random()
        resultMessage = if (dice1 == dice2) {
            "Selamat! Anda dapat dadu double!"
        } else {
            "Anda belum beruntung!"
```

```
        }
    }

    val dice1Image = getDiceImage(dice1)
    val dice2Image = getDiceImage(dice2)

    Box(modifier = Modifier.fillMaxSize()) {
        Column(
            modifier = Modifier.fillMaxSize(),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Row(
                horizontalArrangement = Arrangement.spacedBy(16.dp)
            ) {
                Image(
                    painter = painterResource(id = dice1Image),
                    contentDescription = "Dice 1",
                    modifier = Modifier.size(120.dp)
                )
                Image(
                    painter = painterResource(id = dice2Image),
                    contentDescription = "Dice 2",
                    modifier = Modifier.size(120.dp)
                )
            }

            Spacer(modifier = Modifier.height(32.dp))

            Button(onClick = { rollDice() }) {
                Text(text = "Roll")
            }
        }

        if (resultMessage.isNotEmpty()) {
            Surface(
                color = MaterialTheme.colorScheme.surfaceVariant,
                shape = MaterialTheme.shapes.small,
                modifier = Modifier
                    .fillMaxWidth()
                    .align(Alignment.BottomCenter)
                    .padding(16.dp)
            ) {
                Text(
```

```
                        text = resultMessage,
                        modifier = Modifier.padding(16.dp)
                    )
                }
            }
        }
}

fun getDiceImage(diceValue: Int): Int {
    return when (diceValue) {
        1 -> R.drawable.dice_1
        2 -> R.drawable.dice_2
        3 -> R.drawable.dice_3
        4 -> R.drawable.dice_4
        5 -> R.drawable.dice_5
        6 -> R.drawable.dice_6
        else -> R.drawable.dice_0
    }
}
```

Tabel 2. Source Code MainActivity.kt (XML)

```
package com.example.modul1xml

import android.os.Bundle
import android.view.View
import android.widget.Button
import android.widget.ImageView
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import com.google.android.material.card.MaterialCardView

class MainActivity : AppCompatActivity() {
    private lateinit var imgDice1: ImageView
    private lateinit var imgDice2: ImageView
    private lateinit var btnRoll: Button
    private lateinit var tvResult: TextView
    private lateinit var cardResult: MaterialCardView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        imgDice1 = findViewById(R.id.imgDice1)
        imgDice2 = findViewById(R.id.imgDice2)
        btnRoll = findViewById(R.id.btnRoll)
        tvResult = findViewById(R.id.tvResult)
        cardResult = findViewById(R.id.cardResult)

```

```
28          btnRoll.setOnClickListener {
29              rollDice()
30          }
31      }
32
33      private fun rollDice() {
34          val dice1Value = (1..6).random()
35          val dice2Value = (1..6).random()
36

37  imgDice1.setImageResource(getDiceImage(dice1Value))

38  imgDice2.setImageResource(getDiceImage(dice2Value))
39
40          if (dice1Value == dice2Value) {
41              tvResult.text = "Selamat, anda dapat dadu
    double!"
42          } else {
43              tvResult.text = "Anda belum beruntung!"
44          }
45
46          cardResult.visibility = View.VISIBLE
47      }
48
49      private fun getDiceImage(diceValue: Int): Int {
50          return when (diceValue) {
51              1 -> R.drawable.dice_1
52              2 -> R.drawable.dice_2
53              3 -> R.drawable.dice_3
54              4 -> R.drawable.dice_4
55              5 -> R.drawable.dice_5
56              6 -> R.drawable.dice_6
57              else -> R.drawable.dice_0
58          }
59      }
60  }
```

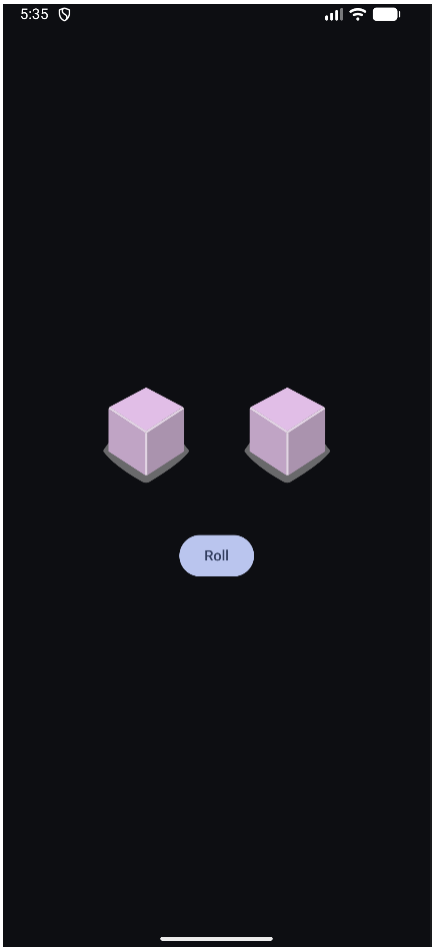
Tabel 3. Source Code activity_main.xml (XML)

```
1   <?xml version="1.0" encoding="utf-8" ?>
2   <androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
3       xmlns:app="http://schemas.android.com/apk/res-auto"
4       xmlns:tools="http://schemas.android.com/tools"
5       android:layout_width="match_parent"
6       android:layout_height="match_parent"
7       tools:context=".MainActivity">
8
9       <LinearLayout
```

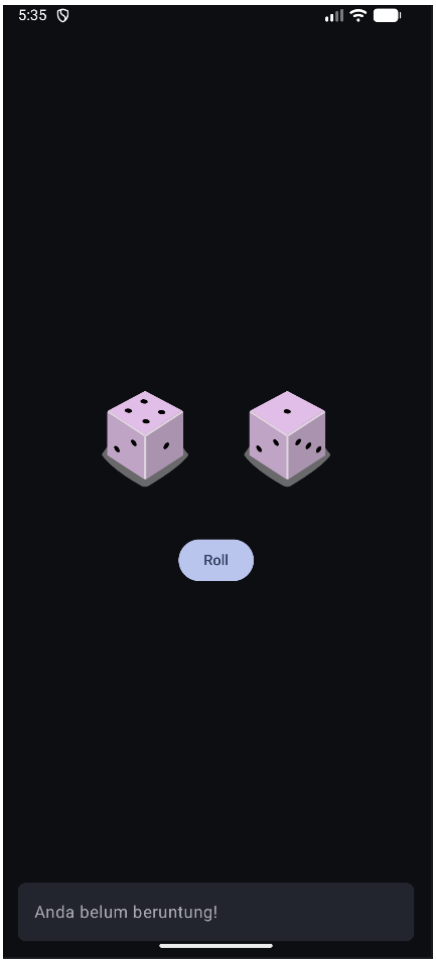
```
10          android:id="@+id/diceContainer"
11          android:layout_width="wrap_content"
12          android:layout_height="wrap_content"
13          android:orientation="horizontal"
14          app:layout_constraintBottom_toTopOf="@+id/btnRoll"
15          app:layout_constraintEnd_toEndOf="parent"
16          app:layout_constraintStart_toStartOf="parent"
17          app:layout_constraintTop_toTopOf="parent"
18          app:layout_constraintVertical_chainStyle="packed">
19
20          <ImageView
21              android:id="@+id/imgDice1"
22              android:layout_width="120dp"
23              android:layout_height="120dp"
24              android:layout_marginEnd="16dp"
25              android:src="@drawable/dice_0"/>
26
27          <ImageView
28              android:id="@+id/imgDice2"
29              android:layout_width="120dp"
30              android:layout_height="120dp"
31              android:src="@drawable/dice_0"/>
32      </LinearLayout>
33
34      <Button
35          android:id="@+id/btnRoll"
36          android:layout_width="wrap_content"
37          android:layout_height="wrap_content"
38          android:layout_marginTop="32dp"
39          android:text="Roll"
40          app:layout_constraintBottom_toBottomOf="parent"
41          app:layout_constraintEnd_toEndOf="parent"
42          app:layout_constraintStart_toStartOf="parent"

43  app:layout_constraintTop_toBottomOf="@+id/diceContainer"/>
44
45      <com.google.android.material.card.MaterialCardView
46          android:id="@+id/cardResult"
47          android:layout_width="match_parent"
48          android:layout_height="wrap_content"
49          android:layout_margin="16dp"
50          android:visibility="gone"

51  app:cardBackgroundColor="?attr/colorSurfaceVariant"
52          app:cardCornerRadius="8dp"
53          app:cardElevation="0dp"
54          app:layout_constraintBottom_toBottomOf="parent"
55          app:layout_constraintEnd_toEndOf="parent"
56          app:layout_constraintStart_toStartOf="parent">
```

```

        <TextView
            android:id="@+id/tvResult"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:padding="16dp"
            android:text="Hasil Roll"

android:textColor="?attr/colorOnSurfaceVariant"/>
    </com.google.android.material.card.MaterialCardView>

</androidx.constraintlayout.widget.ConstraintLayout>
```

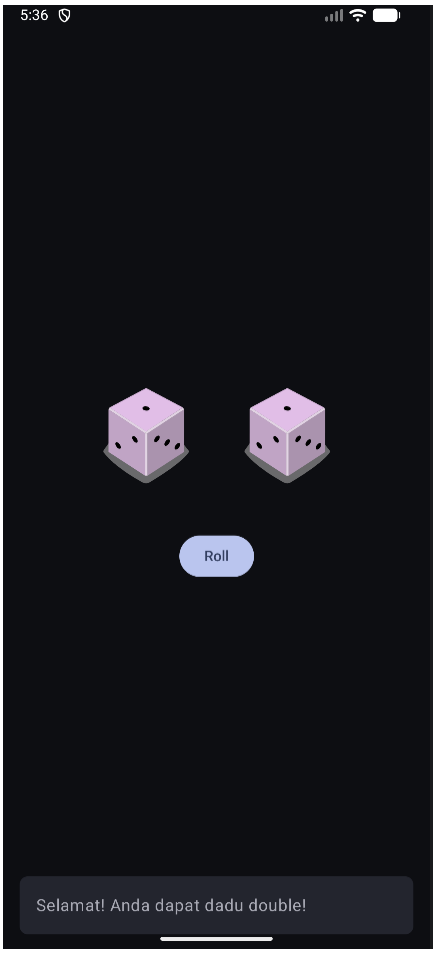
C. Output Program



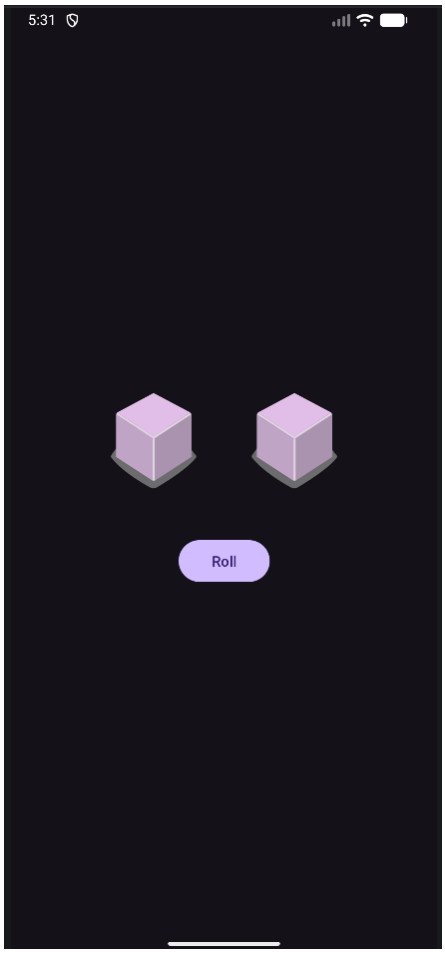
Gambar 4. Screenshot Hasil Jawaban Soal 1 Compose



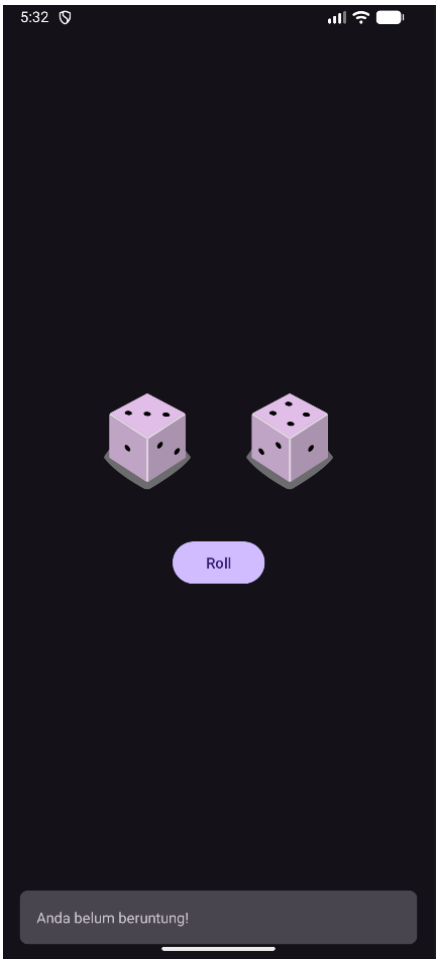
Gambar 5. Screenshot Hasil Jawaban Soal 1 Compose



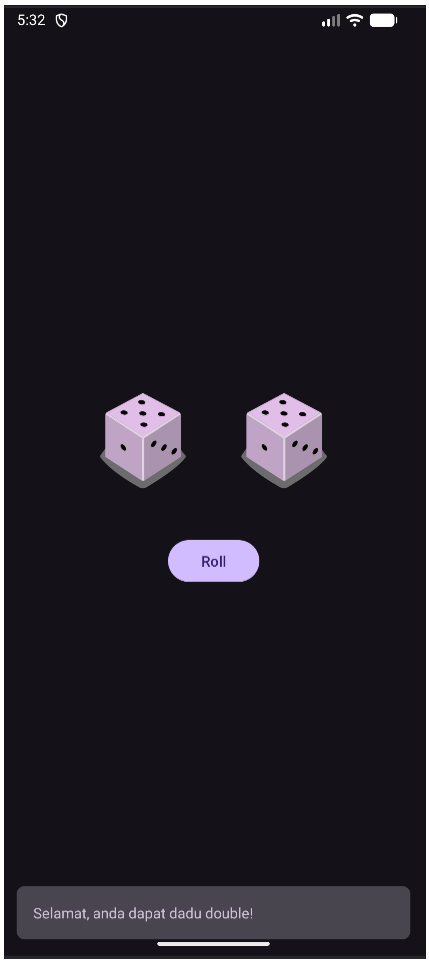
Gambar 6. Screenshot Hasil Jawaban Soal 1 Compose



Gambar 7. Screenshot Hasil Jawaban Soal 1 XML



Gambar 8. Screenshot Hasil Jawaban Soal 1 XML



Gambar 9. Screenshot Hasil Jawaban Soal 1 XML

D. Pembahasan

MainActivity.kt (Compose)

Pada baris [1] sampai [14], `package` digunakan untuk mendeklarasikan direktori file program, sedangkan deretan `import` digunakan untuk memanggil *library* dasar Android dan komponen Jetpack Compose (seperti `Image`, `Column`, `Button`, `Text`) yang dibutuhkan untuk merender UI.

Pada baris [16], `class MainActivity : ComponentActivity()` adalah deklarasi kelas utama. Di arsitektur modern Android, `ComponentActivity` adalah kelas dasar yang wajib digunakan jika ingin membangun UI menggunakan Jetpack Compose.

Pada baris [17] dan [18], `override fun onCreate(savedInstanceState: Bundle?)` adalah metode *lifecycle* yang pertama kali dipanggil saat aplikasi dijalankan.

Pada baris [19], `setContent {…}` adalah blok fungsi khusus milik Compose. Semua fungsi UI Composable harus dipanggil dari dalam blok ini agar bisa ddigambar ke layar perangkat.

Pada baris [20] sampai [26], `Modul1ComposeTheme` merupakan tema warna bawaan. Di dalamnya terdapa `Surface` yang bertindak sebagai kanvas atau latar belakang utama. Modifier.fillMaxSize membuat kanvas tersebut memenuhi seluruh layer, dan `DiceRollerApp()` dipanggil untuk mulai merender tampilan aplikasi utama.

Pada baris [32], `@Composable` adalah anotasi yang memberitahu *compiler* Kotlin bahwa fungsi di bawahnya adalah komponen UI yang bisa digambar oleh Jetpack Compose.

Pada baris [30] sampai [32], `var dice1 by remember { mutableInsStateOf(0) }` serta variabel `dice2` dan `resultMessage` adalah mekanisme penyimpanan *State* di Compose. `mutableStateOf` bertugas memantau perubahan data dan otomatis memicu render ulang (*recomposition*) pada UI jika datanya berubah. Fungsi `remember` memastikan data ini tidak kembali ke nol saat proses render ulang terjadi.

Pada baris [33] sampai [46], `fun rollDice` adalah fungsi lokal yang menyimpan aturan permainan. Fungsi ini mengacak angka menggunakan `(1..6).random()` untuk kedua dadu. Jika angka dadu 1 dan 2 sama, pesan "Selamat, anda dapat dadu double!" dimasukkan ke dalam `resultMessage`. Jika berbeda, pesannya jadi "Anda belum beruntung!".

Pada baris [48] dan [49], variabel `dice1Image` dan `dice2Image` memanggil fungsi pembantu `getDiceImage()` untuk menerjemahkan angka dadu yang terpilih menjadi ID berkas gambar yang tepat.

Pada baris [51], `Box(modifier = Modifier.fillMaxSize())` adalah *container* bertipe *Box* yang mengisi seluruh layar. *Box* memungkinkan elemen di dalamnya saling tumpang tindih atau ditempatkan di posisi spesifik.

Pada baris [52] sampai [55], `Column` menyusun elemen-elemen di dalamnya secara vertikal dari atas ke bawah. Pengaturan `Alignment.CenterHorizontally` dan `Arrangement.Center` memastikan konten berada tepat di tengah-tengah layar.

Pada baris [57] sampai [70], `Row` menyusun komponen secara horizontal dengan jarak spasi 16dp. Di dalamnya terdapat dua komponen `Image` yang menampilkan gambar dadu menggunakan `painterResource` dan diberi ukuran konstan 120dp.

Pada baris [72], `Spacer` memberikan jarak kosong vertikal setinggi 32dp untuk memisahkan gambar dadu dengan tombol “Roll”.

Pada baris [74] sampai [76], `Button` mendefinisikan sebuah tombol. Pada bagian `onClick = { rollDice() }`, itu menginstruksikan tombol tersebut untuk menjalankan fungsi acak dadu setiap kali ditekan. Teks pada tombol didefinisikan menggunakan komponen `Text ("Roll")`.

Pada baris [79], `if (resultMessage.isNotEmpty())` adalah rendering kondisional. Blok UI di dalamnya hanya akan digambar dan muncul di layar jika variabel pesan hasil tidak kosong.

Pada baris [80] sampai [91], `Surface` digunakan sebagai kotak latar untuk teks hasil. Atribut pendukungnya adalah `shape` untuk membuat sudut membulat, `fillMaxWidth` untuk memanjangkan kotak, dan `.align(Alignment.BottomCenter)` yang berfungsi menempelkan kotak

pesan ini di area paling bawah layar. Teks pesannya sendiri dirender oleh komponen `Text` di dalam `Surface` tersebut.

Pada baris [97] sampai [107], `fun getDiceImage(diceValue: Int): Int` adalah fungsi biasa yang bukan Composable dan menerima nilai angka bulat (integer). Fungsi ini menggunakan blok `when` yang mengganti switch-case di Java, fungsinya untuk mencocokkan angka dengan *resource id* gambar yang ada di folder drawable. Bagian `else -> R.drawable.dice_0` memastikan bahwa saat aplikasi pertama kali dibuka, gambar dadu kosong yang tampil.

MainActivity.kt (XML)

Pada baris [1] sampai [9], `package` digunakan untuk mendeklarasikan direktori file program, sedangkan deretan `import` digunakan untuk memanggil *library* antarmuka Android klasik (seperti `View`, `Button`, `ImageView`, `TextView`) dan komponen Material Design (`MaterialCardView`) yang dibutuhkan untuk berinteraksi dengan elemen XML.

Pada baris [11], `class MainActivity : ComponentActivity()` adalah deklarasi kelas utama. Berbeda dengan Compose, aplikasi Android berbasis XML wajib mewarisi kelas `AppCombatActivity` agar bisa merender tampilan XML dan mendukung fitur-fitur kompatibilitas perangkat Android versi lama.

Pada baris [12] sampai [16], dideklarasikan variabel untuk menampung komponen UI. Kata kunci `private lateinit var` (*late initialization*) digunakan agar bisa mendeklarasikan variabelnya sekarang, tapi baru akan mengisinya dengan komponen UI nanti (saat aplikasi dibuat di memori), sehingga terhindar dari error null.

Pada baris [18] dan [19], `override fun onCreate(savedInstanceState: Bundle?)` adalah metode *lifecycle* utama yang pertama kali dijalankan saat aplikasi dibuka.

Pada baris [20], `setContentView(R.layout.activity_main)` adalah perintah yang berfungsi untuk memuat dan menyambungkan desain antarmuka yang

ada di file `activity_main.xml` ke dalam kelat Kotlin ini agar bisa ditampilkan ke layar.

Pada baris [22] sampai [26], metode `findViewById(R.id…)` digunakan secara berurutan. Ini adalah proses pencarian di mana sistem Android mencari elemen UI di dalam XML berdasarkan ID-nya, lalu mengaitkannya (*binding*) ke variabel Kotlin yang sudah dibuat sebelumnya.

Pada baris [28] sampai [30], `btnRoll.setOnClickListener {…}` berfungsi untuk memasang *event listener* pada tombol "Roll". Jika tombol tersebut diklik oleh pengguna, maka baris perintah di dalamnya, yaitu memanggil fungsi `rollDice()`, akan dieksekusi.

Pada baris [33] sampai [47], `private fun rollDice()` adalah fungsi lokal yang menyimpan aturan permainan, kata kunci `private` memastikan fungsi ini hanya bisa dipanggil dari dalam `MainActivity` ini saja. Fungsi ini mengacak angka menggunakan `(1..6).random()` untuk kedua dadu, lalu memperbarui gambarnya. Jika angka dadu 1 dan 2 sama, pesan "Selamat, anda dapat dadu double!" ditampilkan pada `tvResult`. Jika berbeda, pesannya menjadi "Anda belum beruntung!". Terakhir, `cardResult.visibility = View.VISIBLE` adalah perintah yang sangat penting. Di XML, kartu pesan disembunyikan secara *default* (`visibility= "gone"`). Perintah ini bertugas untuk memunculkan kartu pesan tersebut ke layar setelah tombol Roll ditekan pertama kalinya.

Pada baris [49] sampai [59], `private fun getDiceImage(diceValue: Int): Int` adalah fungsi pembantu yang menerima nilai angka bulat (integer). Fungsi ini menggunakan blok `when`, fungsinya untuk mencocokkan angka dengan *resource id* gambar yang ada di folder drawable. Bagian `else -> R.drawable.dice_0` memastikan bahwa saat aplikasi pertama kali dibuka, gambar dadu kosong yang tampil.

activity_main.xml (XML)

Pada baris [2] sampai [7], ConstraintLayout adalah tata letak dasar (*root layout*) yang menampung seluruh elemen antarmuka. Atribut `match_parent` digunakan agar layout ini memenuhi seluruh ukuran panjang dan lebar layar perangkat.

Pada baris [9] sampai [33], `LinearLayout` digunakan sebagai wadah untuk Menyusun dua buah dadu secara horizontal menggunakan `orientation="horizontal"`. Di dalamnya terdapat dua komponen `ImageView` (`imgDice1` dan `imgDice2`) yang masing-masing diberi ukuran tetap 120dp dan gambar bawaan berupa dadu kosong (`@drawable/dice_0`).

Pada baris [34] sampai [43], komponen `Button` dengan ID `btnRoll` digunakan untuk merender tombol bertuliskan "Roll". Tombol ini diposisikan berada di tengah layar tepat di bawah wadah dadu menggunakan bantuan atribut `app:layout_constrain....`

Pada baris [45] sampai [65], komponen `MaterialCardView` digunakan untuk membungkus sebuah `TextView` (`tvResult`) yang bertugas menampilkan pesan hasil akhir. Atribut `android:visibility= "gone"` digunakan agar kartu pesan ini disembunyikan dari layar saat aplikasi pertama kali dijalankan. Posisinya diatur agar menempel di bagian paling bawah layar menggunakan `layout_constraintBottom_toBottomOf="parent"`.

E. Essay

Dalam dunia rekayasa perangkat lunak, paradigma pemrograman menentukan bagaimana sebuah masalah dipecahkan dan bagaimana struktur kode diorganisasikan. Pemrograman imperatif merupakan paradigma di mana *state* dimodifikasi melalui pernyataan eksplisit, berfokus pada instruksi mendetail mengenai bagaimana sebuah masalah harus diselesaikan [1]. Dalam pendekatan ini, *developer* memberikan langkah-langkah spesifik untuk memanipulasi setiap komponen antarmuka pengguna (UI). Sebaliknya, pemrograman deklaratif berfokus pada deskripsi apa yang harus

ditampilkan dan hasil akhir yang diharapkan, sementara sistem internal dari *framework* yang akan mengatur mekanisme teknis untuk mencapai hasil tersebut [1].

Penerapan kedua paradigma ini sangat kentara dan terus berevolusi dalam pengembangan antarmuka aplikasi *mobile* modern. Pada ekosistem Android, pendekatan imperatif tradisional diimplementasikan menggunakan bahasa Kotlin atau Java yang dipadukan dengan tata letak berbasis XML [1]. Sementara itu, pergeseran menuju paradigma deklaratif pada platform yang sama diwujudkan melalui Jetpack Compose, di mana UI dapat mendeskripsikan elemennya sendiri dan beradaptasi secara otomatis mengikuti *state* data terkini [1]. Di sisi lain, pada platform iOS, *framework* UIKit menjadi representasi utama dari arsitektur imperatif. Dalam UIKit, *developer* memikul tanggung jawab penuh untuk mengelola siklus hidup antarmuka dan melakukan sinkronisasi tampilan secara manual melalui mekanisme *event-driven* [2]. Sebagai inovasi, Apple kemudian memperkenalkan SwiftUI yang mengusung arsitektur deklaratif *state-driven*, yang memungkinkan UI untuk dirender ulang secara otomatis tanpa intervensi manual ketika terjadi perubahan data [2].

Berdasarkan tinjauan literatur empiris tersebut, secara personal saya lebih memilih pendekatan hibrida yang disesuaikan dengan platform target, yaitu memprioritaskan paradigma imperatif untuk Android dan paradigma deklaratif untuk iOS. Preferensi ini didasarkan pada bukti metrik efisiensi dan performa operasional. Di lingkungan Android, pendekatan imperatif berbasis XML terbukti lebih unggul dalam memberikan stabilitas *rendering* visual dan manajemen memori yang efisien [1]. Pengujian *benchmarking* menunjukkan bahwa aplikasi Kotlin-XML mencatat waktu *startup* awal (*cold start*) yang lebih cepat, serta mampu menjaga latensi yang konsisten dengan tingkat *frame drops* (keterlambatan *render*) yang sangat minim saat menangani beban komputasi berat, seperti navigasi antarlayar atau *scrolling* [1].

Sebaliknya, pada ekosistem iOS, adopsi paradigma deklaratif melalui SwiftUI menawarkan keunggulan produktivitas dan skalabilitas arsitektur yang mutlak. Secara kuantitatif, implementasi antarmuka deklaratif berhasil memangkas waktu pengembangan sebesar 36,8% dan mengurangi ukuran kode hingga 40% dibandingkan

dengan pendekatan imperatif UIKit [2]. Dengan mendelegasikan sinkronisasi *state* sepenuhnya kepada *framework*, *developer* terbebas dari pengelolaan siklus hidup UI yang rumit, yang secara langsung meminimalisir kemunculan *bug* inkonsistensi visual. Mengingat performa *runtime* SwiftUI tetap setara dengan UIKit, pergeseran ke arah deklaratif di iOS bukan sekadar tren, melainkan solusi efisiensi yang krusial untuk pengembangan perangkat lunak modern [2].

Meskipun Kotlin beroperasi di atas *Java Virtual Machine* (JVM) dan dirancang untuk memiliki interoperabilitas penuh dengan Java, terdapat perbedaan filosofis yang fundamental dalam implementasi *Object-Oriented Programming* (OOP) antara kedua bahasa tersebut [3]. Perbedaan paling mencolok terletak pada pendekatan *inheritance*. Dalam Java, sebuah kelas secara bawaan dapat diwariskan kecuali ditandai secara eksplisit dengan kata kunci final. Sebaliknya, Kotlin menerapkan filosofi desain yang lebih ketat untuk menghindari *fragile base class problem*, oleh karena itu, seluruh kelas dan metode di Kotlin bersifat final secara *default*. Untuk mengizinkan pewarisan, *developer* harus menambahkan pengubah open secara eksplisit [3].

Perbedaan lainnya mencakup pengelolaan status dan anggota kelas. Kotlin menyederhanakan deklarasi kelas dengan memperkenalkan konsep *Properties* yang menggantikan kebutuhan penulisan metode *getter* dan *setter* yang panjang seperti di Java [3]. Selain itu, Kotlin tidak memiliki kata kunci static untuk mendeklarasikan anggota atau fungsi pada level kelas. Sebagai gantinya, Kotlin menggunakan pendekatan pola desain seperti *companion object* atau mendeklarasikan *top-level functions* untuk mencapai tujuan yang sama dengan cara yang lebih berorientasi pada objek [3]. Terakhir, *nested classes* di Kotlin tidak menyimpan referensi ke kelas luarnya secara otomatis seperti *inner class* di Java, melainkan berperilaku layaknya *static nested class* di Java, kecuali jika ditambahkan kata kunci inner [3].

Untuk menjaga struktur perangkat lunak yang dibangun dengan paradigma OOP agar tetap dapat dipelihara dan terukur, *developer* mengacu pada lima prinsip desain fundamental yang dikenal dengan SOLID. Prinsip pertama adalah *Single Responsibility Principle* (SRP), yang menyatakan bahwa sebuah modul atau kelas

hanya boleh memiliki satu alasan untuk berubah [4]. Artinya, sebuah entitas perangkat lunak harus berfokus pada satu tanggung jawab atau tugas spesifik, sehingga perubahan pada salah satu fitur tidak memengaruhi atau merusak fungsionalitas lain yang tidak terkait. Selanjutnya, *Open-Closed Principle* (OCP) mensyaratkan bahwa artefak perangkat lunak harus bersifat *open for extension*, namun *closed for modification* [4]. Hal ini mengimplikasikan bahwa penambahan fungsionalitas baru harus dilakukan dengan menambahkan kode baru, misalnya melalui abstraksi atau pewarisan tanpa mengubah kode inti yang sudah ada dan berjalan stabil.

Prinsip ketiga, *Liskov Substitution Principle* (LSP), menekankan bahwa objek dari sub-kelas harus dapat menggantikan objek dari kelas induknya tanpa merusak kebenaran dan logika aplikasi [4]. Sub-kelas wajib mematuhi kontrak dari abstraksinya dan tidak boleh mengubah perilaku fundamental yang diharapkan dari kelas dasar tersebut. Kemudian, terdapat *Interface Segregation Principle* (ISP) yang menyarankan agar klien tidak dipaksa untuk bergantung pada metode antarmuka yang tidak mereka gunakan [4]. Praktik terbaik dari prinsip ini adalah memecah antarmuka yang besar menjadi beberapa antarmuka yang lebih kecil dan kohesif untuk melayani kebutuhan klien secara spesifik. Terakhir, *Dependency Inversion Principle* (DIP) menetapkan bahwa modul tingkat tinggi tidak boleh bergantung pada modul tingkat rendah yang berisi detail implementasi [4]. Sebaliknya, keduanya harus bergantung pada abstraksi, di mana implementasi detail pada akhirnya harus menyesuaikan dengan abstraksi tersebut.

Dalam Kotlin, pembeda utama antara kelas reguler (class) dan data class terletak pada otomatisasi pembuatan kode berulang (*boilerplate*) oleh kompilator yang berfokus pada manajemen data [3]. Kelas reguler dirancang sebagai wadah konvensional untuk merepresentasikan objek yang memiliki *state* dan *behavior*. Jika *developer* ingin membandingkan nilai antar objek reguler atau mencetak nilainya ke dalam format string yang informatif, mereka diwajibkan untuk *override* metode bawaan seperti equals(), hashCode(), dan toString() secara manual [3].

Sebaliknya, data class diciptakan secara spesifik dengan tujuan utama sebagai struktur pembawa data (seperti *Data Transfer Object*). Hanya dengan memberi kata kunci data di depan deklarasi kelas, kompilator Kotlin secara otomatis menghasilkan metode utilitas esensial, yaitu equals(), hashCode(), dan toString() berdasarkan properti-properti yang dideklarasikan di dalam konstruktor utamanya [3]. Tidak hanya itu, data class juga secara otomatis dibekali dengan metode copy() yang memudahkan *developer* untuk menduplikasi objek sekaligus mengubah properti tertentu, sehingga sangat mendukung praktik imutabilitas (objek yang tidak dapat diubah setelah dibuat) yang aman dalam pemrograman modern [3].

F. Daftar Pustaka

[1] N. Nurhadi dan A. Muslim, "Comparative Analysis of the Impact of Declarative and Imperative Programming Approaches on Android Application Performance Through Benchmarking Method," *JRSSEM (Journal Research of Social Science, Economics, and Management),* vol. 5, no. 5, pp. 5234-5248, 2025.

[2] M. L. Gondi, "A Comparative Study on the Adoption of Swiftui Over Uikit in Modern Ios Application Development," *International Journal of Research and Scientific Innovation (IJRSI),* vol. 13, no. 1, pp. 2425-2432, 2026.

[3] S. Aigner, R. Elizarov, S. Isakova dan D. Jemerov, Kotlin in Action, Second Edition, Manning Publications, 2024.

[4] R. C. Martin, Clean Architecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall, 2018.

TAUTAN GIT

https://github.com/lisaryuna/Praktikum-Mobile